

Pipeline Hazard Simulation Applet Report

CS2011: Foundations of Computer Systems

Lab Assignment II

Rohan Gupta

Esraaj Sarkar Gupta

April 26, 2026

1 Objective

This project implements an interactive pipeline hazard simulation applet for a single-issue, in-order processor model. The applet allows a user to enter a short sequence of assembly-like instructions, select a 4-stage or 5-stage pipeline, optionally enable data forwarding, and observe the pipeline schedule cycle by cycle.

The focus of the simulator is RAW (Read After Write) data hazards. The simulator does not model branch prediction, control hazards, structural hazards, caches, exceptions, interrupts, out-of-order execution, superscalar issue, or multi-cycle functional units.

2 How to Run the Applet

The submission is a static web app. To run it, open `index.html` in a browser from the project folder. The main source files are:

- `index.html`: Defines the app layout and controls.
- `style.css`: Defines the visual design, sticky instruction pane, pipeline table styling, tooltips, and forwarding arrows.
- `pipeline.js`: Contains the parser, pipeline timing model, RAW hazard detection, stall insertion, and forwarding event generation.
- `ui.js`: Connects the simulator state to the browser UI, including stepping, auto-run, reset, hazard logs, forwarding logs, tooltips, and table rendering.

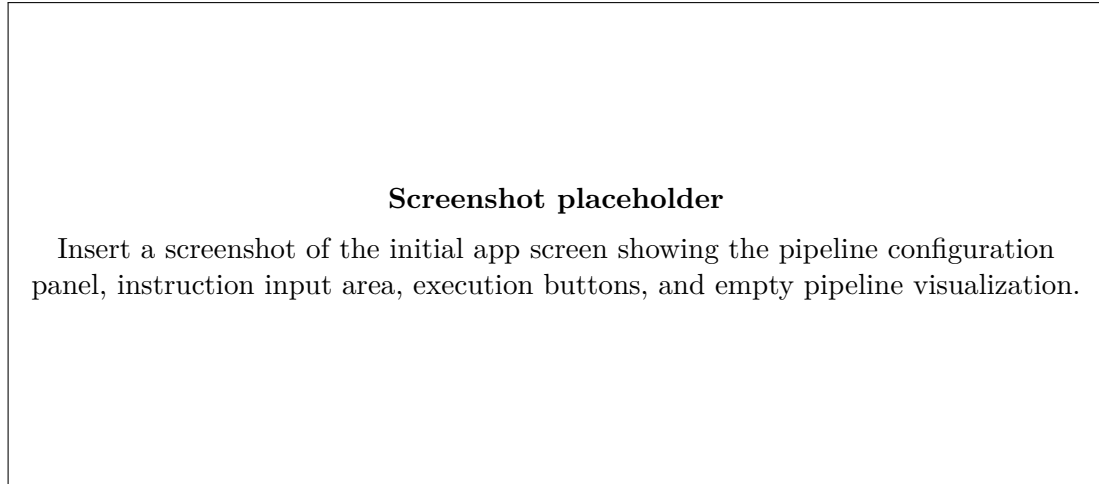


Figure 1: Initial app interface.

3 Instruction Input

The applet accepts at most 10 non-empty instructions per simulation. Instructions are entered one per line in the input box. If any line cannot be parsed, the full input is rejected and no partial program is ingested. The error log reports the original line number and the reason the line failed to parse.

The accepted instruction formats are:

Instruction type	Format
Arithmetic register	ADD rd, rs, rt
Arithmetic register	SUB rd, rs, rt
Arithmetic register	AND rd, rs, rt
Arithmetic register	OR rd, rs, rt
Immediate arithmetic	ADDI rt, rs, imm
Immediate arithmetic	ORI rt, rs, imm
Load	LW rt, offset(base)
Store	SW rt, offset(base)

Registers must use the form R<number>, for example R1, R2, or R10. Immediates and memory offsets must be integers. The assignment requires ADD, SUB, LW, and SW; the applet also supports AND, OR, ADDI, and ORI.

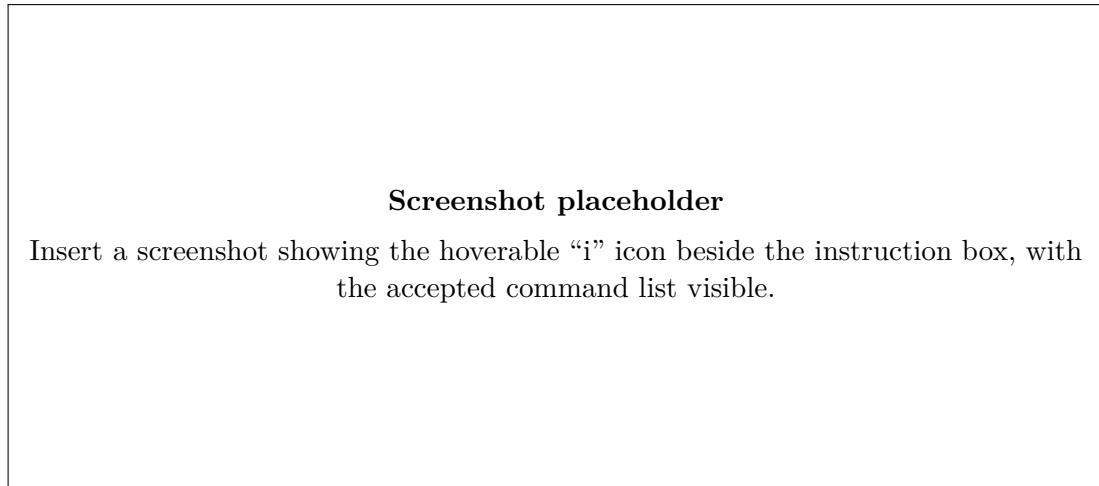


Figure 2: Accepted instruction formats shown in the UI.

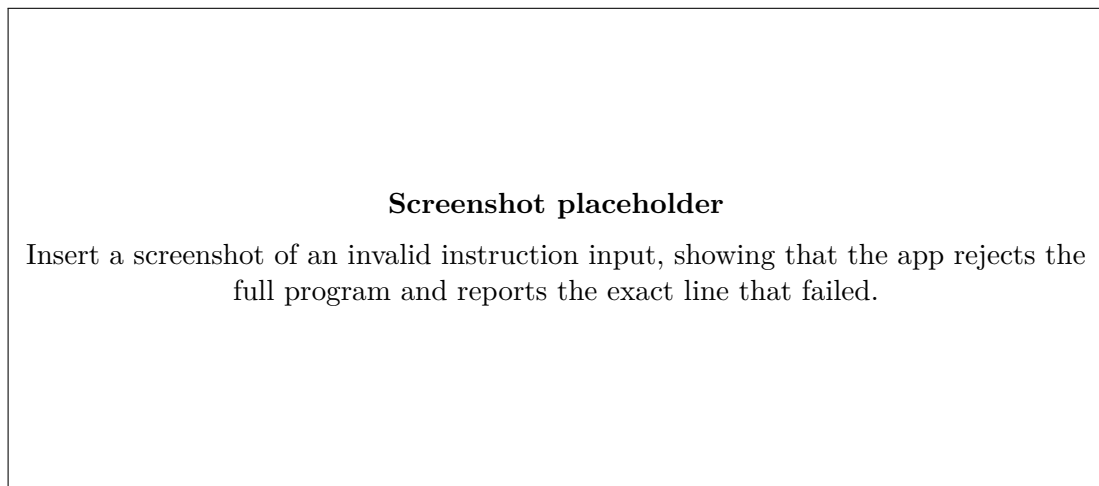


Figure 3: Invalid input handling.

4 Pipeline Configurations

The applet supports two pipeline models:

- **5-stage pipeline:** IF, ID, EX, MEM, WB.
- **4-stage pipeline:** IF, ID, EX, MEM/WB. In this model, memory access and write-back are represented as one combined completion stage.

The selected pipeline changes the produced schedule. In the 4-stage pipeline, instructions complete earlier because MEM and WB are combined into MEM/WB.

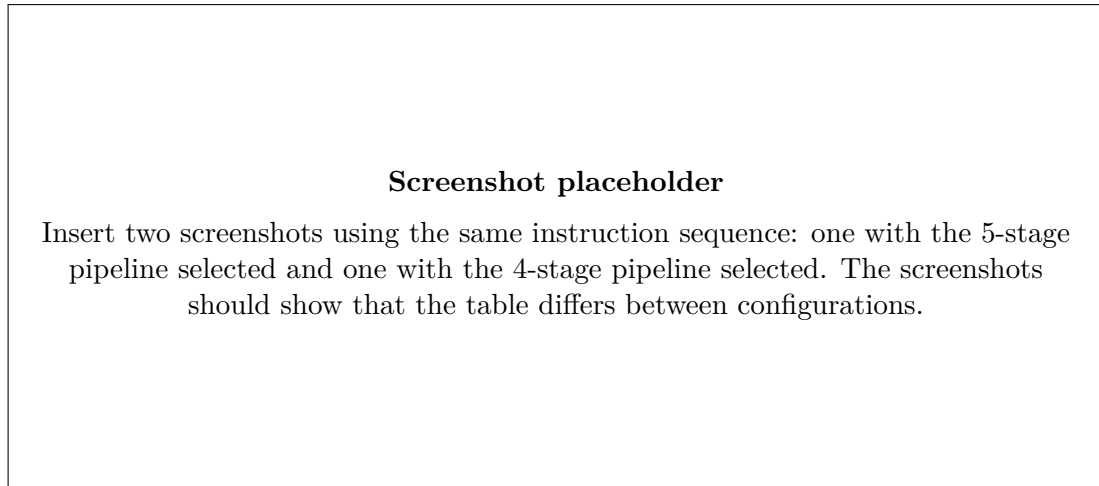


Figure 4: Comparison of 5-stage and 4-stage pipeline configurations.

5 Pipeline Visualization and Interaction

The table displays instructions as rows and cycles as columns. Each populated cell shows the stage occupied by that instruction during that cycle. Empty cells indicate that the instruction has not yet started or has already completed.

The instruction column remains visible while horizontally scrolling through cycles. The title “Pipeline Visualization” stays attached to the top of the visualization card rather than scrolling inside the table. The table grows only as cycles are rendered, so during step execution or auto-run the user can scroll only as far as the currently visible clock cycle.

The applet supports:

- **Step:** advances the schedule by one cycle.
- **Auto-Run:** advances one cycle every 600 ms until completion.
- **Reset Run:** resets the current loaded program to cycle 0.
- **Clear:** clears the input and loaded simulation.

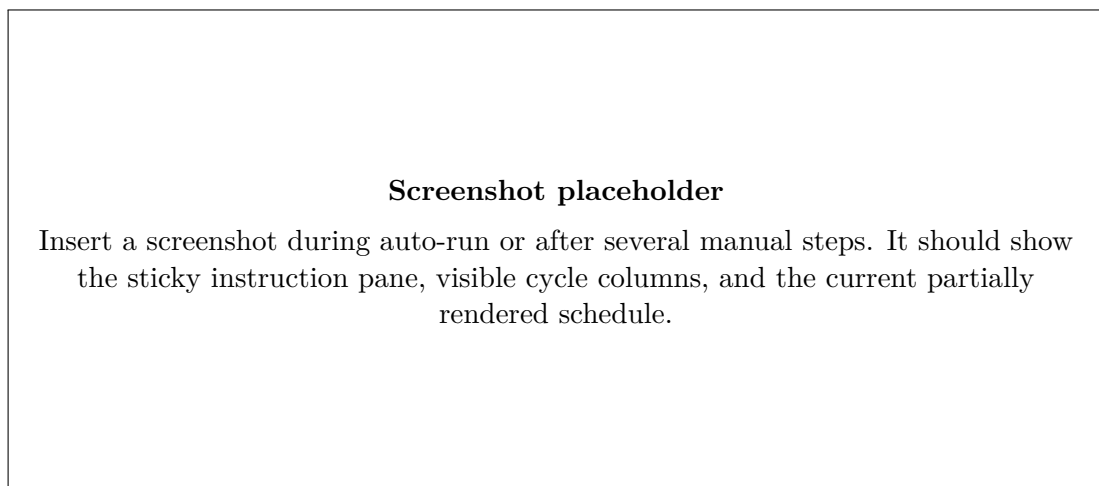


Figure 5: Cycle-by-cycle pipeline table.

6 Hazard Detection

The simulator detects RAW hazards. For each instruction, the simulator checks the latest prior instruction that writes each source register. If a source register depends on an earlier result that is not yet available under the selected model, the consumer instruction is delayed by inserting explicit **ST** stall cells.

WAR and WAW hazards are not modeled as baseline hazards because the simulator is single-issue and in-order. Control and structural hazards are also outside the simulator scope, matching the assignment requirements.

When a stall appears, the visible cell says which register is being waited on, for example **Waiting for R1**. Hovering over the stall gives a more detailed explanation, such as which instruction is expected to produce that register value.

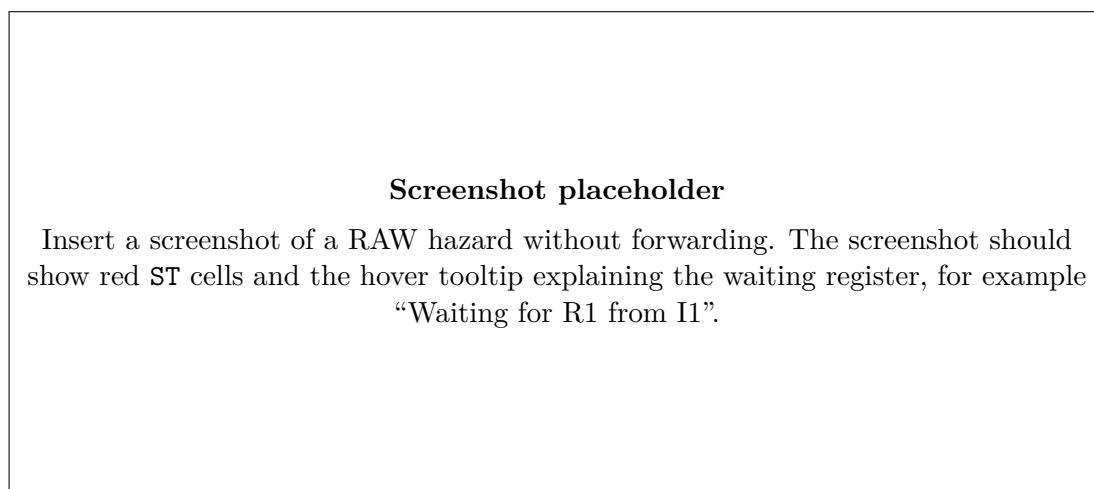


Figure 6: RAW hazard resolved through stalls.

7 Forwarding Model

When forwarding is enabled, the simulator generates forwarding events for RAW dependencies that can be satisfied through bypassing. The UI highlights the source and target cells and draws a forwarding arrow. Hovering over an arrow shows which register is being forwarded and the source and destination stages.

The forwarding model assumes a standard in-order MIPS-style datapath:

- ALU results may be forwarded to a later **EX** input.
- Load data is available only after the load’s memory stage, so a direct load-use dependency still requires one stall before forwarding.
- Store data is consumed in the memory stage, so store-data dependencies can forward to that stage.
- If a producer reaches **WB** or **MEM/WB** in the same cycle that a consumer is in **ID**, the value is treated as readable from the register file, so no unnecessary forwarding arrow is drawn for that case.
- If multiple earlier instructions write the same register, the simulator uses the latest prior writer for the dependency, avoiding stale forwarding.

Screenshot placeholder

Insert a screenshot of an arithmetic RAW dependency with forwarding enabled. It should show fewer or no stalls compared with the no-forwarding version and should show a forwarding arrow.

Figure 7: Forwarding for an arithmetic RAW dependency.

Screenshot placeholder

Insert a screenshot of a load-use dependency with forwarding enabled. It should show exactly one stall before the consumer proceeds, plus a forwarding arrow from the load result.

Figure 8: Load-use hazard with forwarding.

8 Implementation Structure

8.1 pipeline.js

`pipeline.js` contains the core simulation logic. It defines:

- **Instruction:** stores parsed instruction fields, source registers, destination register, schedule, visible stages, timing, and stall reasons.
- **PipelineSimulator:** stores the loaded instructions, current cycle, pipeline type, forwarding mode, hazards, forwarding events, and parse errors.

Important methods include:

- **loadInstructions:** validates all input lines before accepting the program.
- **parseInstruction:** parses and validates supported instruction syntax.
- **getLatestRawDependencies:** finds the most recent prior writer for each source register.
- **buildSchedule:** computes instruction timing, stalls, hazard messages, and forwarding events.

- `createSchedule`: converts timing into the visible cycle-by-cycle stage row.
- `step`, `runToEnd`, and `resetRun`: control visible simulation progress.

8.2 `ui.js`

`ui.js` renders the simulator state. It builds the table headers and cells for visible cycles only, renders hazard and forwarding logs, draws forwarding arrows, manages the sticky instruction shield, and implements auto-run as a 600 ms cycle-by-cycle loop.

8.3 `style.css`

`style.css` handles the visual presentation. It defines the dark interface, stage cell colors, stall styling, forwarding highlights, sticky instruction pane, forwarding arrow overlay, and responsive behavior.

9 Sample Test Cases

The applet includes preset test cases for both pipeline configurations. The preset list changes automatically when the user selects the 4-stage or 5-stage pipeline. These presets are based on the provided pipeline test case PDFs, converted into the applet's accepted `R<number>` register syntax.

9.1 4-Stage Pipeline Test Cases

When the 4-stage pipeline is selected, the preset dropdown contains the following six test cases:

Preset	Program
TC1: ALU RAW	ADD R1, R2, R3 SUB R4, R1, R5
TC2: ALU RAW with gap	ADD R1, R2, R3 AND R6, R7, R8 SUB R4, R1, R5
TC3: Load-use RAW	LW R1, 0(R2) ADD R3, R1, R4
TC4: Load-use with gap	LW R1, 0(R2) ORI R6, R7, 7 ADD R3, R1, R4
TC5: Chained overwrite RAW	ADD R1, R2, R3 ADD R1, R1, R4 SUB R5, R1, R6
TC6: Store data RAW	ADD R1, R2, R3 SW R1, 0(R4)

4-Stage Evidence Screenshots

Screenshot placeholder

Insert a screenshot of 4-stage TC1 with forwarding disabled. The screenshot should show SUB stalling while waiting for R1.

Figure 9: 4-stage TC1 without forwarding.

Screenshot placeholder

Insert a screenshot of 4-stage TC1 with forwarding enabled. The screenshot should show the forwarding arrow and the shorter schedule compared with the no-forwarding screenshot.

Figure 10: 4-stage TC1 with forwarding.

Screenshot placeholder

Insert a screenshot of 4-stage TC3 with forwarding enabled. The screenshot should show the load-use stall and the visible `Waiting for R1` stall label.

Figure 11: 4-stage TC3 load-use with forwarding.

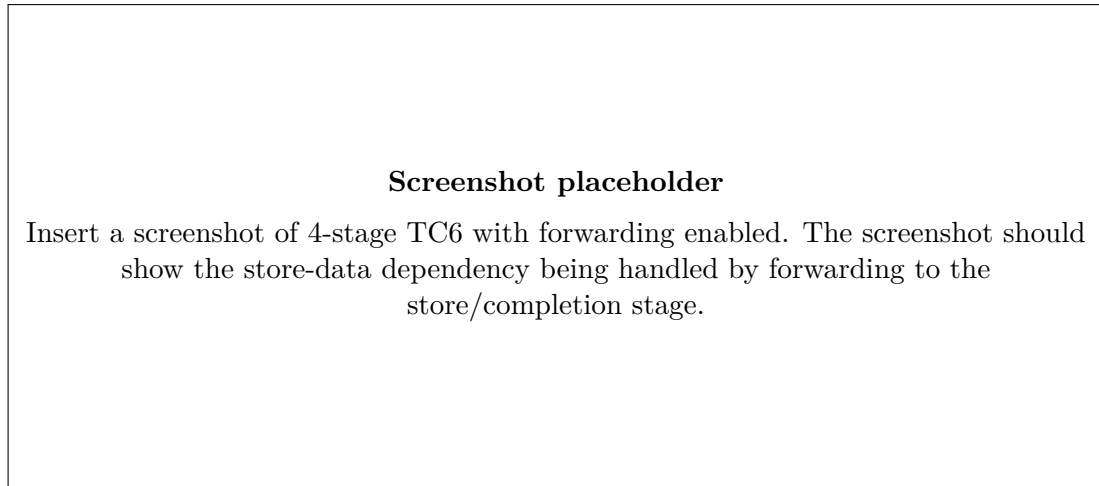


Figure 12: 4-stage TC6 store-data forwarding.

9.2 5-Stage Pipeline Test Cases

When the 5-stage pipeline is selected, the preset dropdown contains the following seven test cases:

Preset	Program
TC1: No dependency	ADD R1, R2, R3
	SUB R4, R5, R6
TC2: ALU RAW	ADD R1, R2, R3
	SUB R4, R1, R5
TC3: Load-use RAW	LW R1, 0(R2)
	ADD R3, R1, R4
TC4: Store data with gap	ADD R1, R2, R3
	ADD R4, R5, R6
	SW R1, 0(R7)
TC5: Load and ALU chain	LW R1, 0(R2)
	ADD R3, R1, R4
	SUB R5, R3, R6
TC6: Load-add-store chain	LW R8, 4(R2)
	ADD R9, R8, R6
	SW R9, 0(R10)
TC7: Longer mixed chain	LW R1, 0(R2)
	ADD R3, R1, R4
	SUB R5, R3, R6
	LW R7, 4(R2)

5-Stage Evidence Screenshots

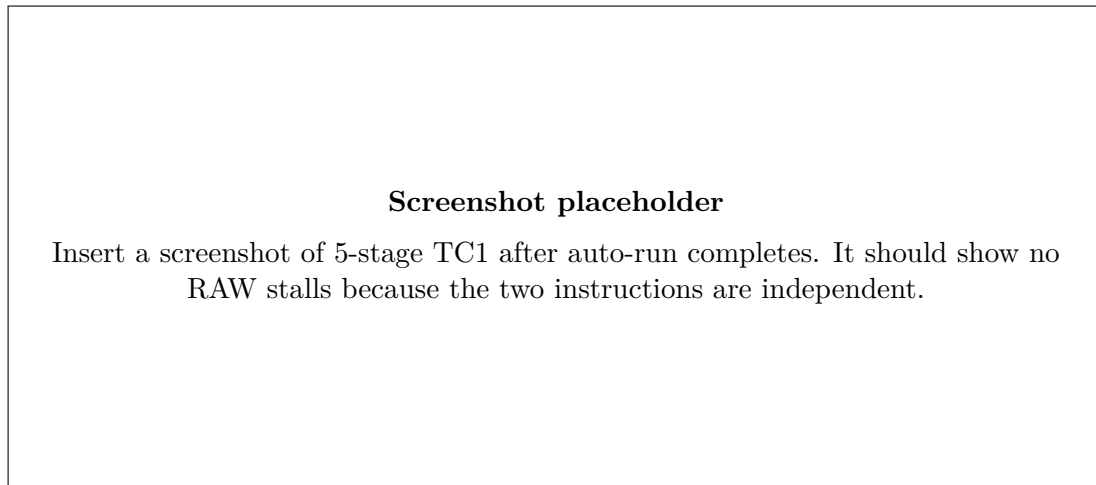


Figure 13: 5-stage TC1 no-dependency case.

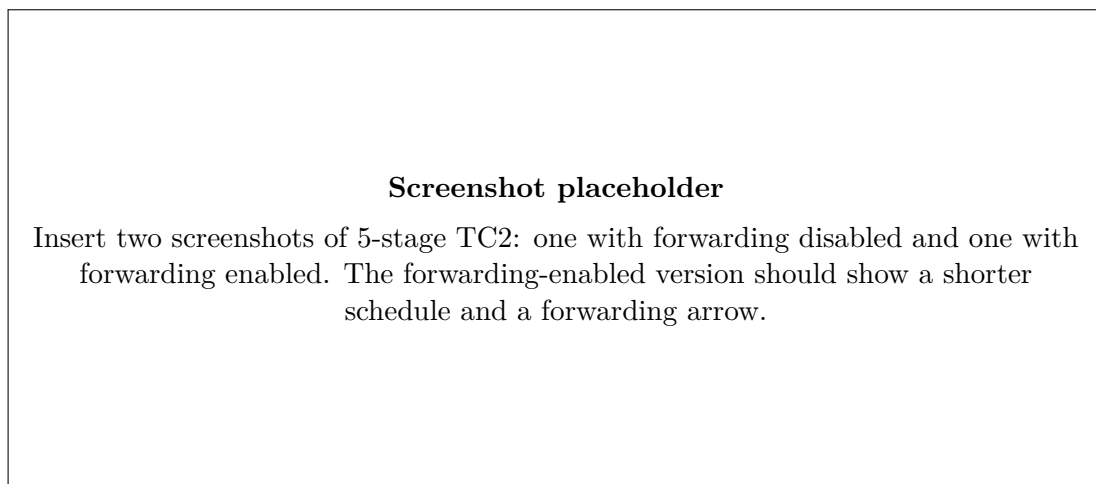


Figure 14: 5-stage TC2 arithmetic RAW with and without forwarding.

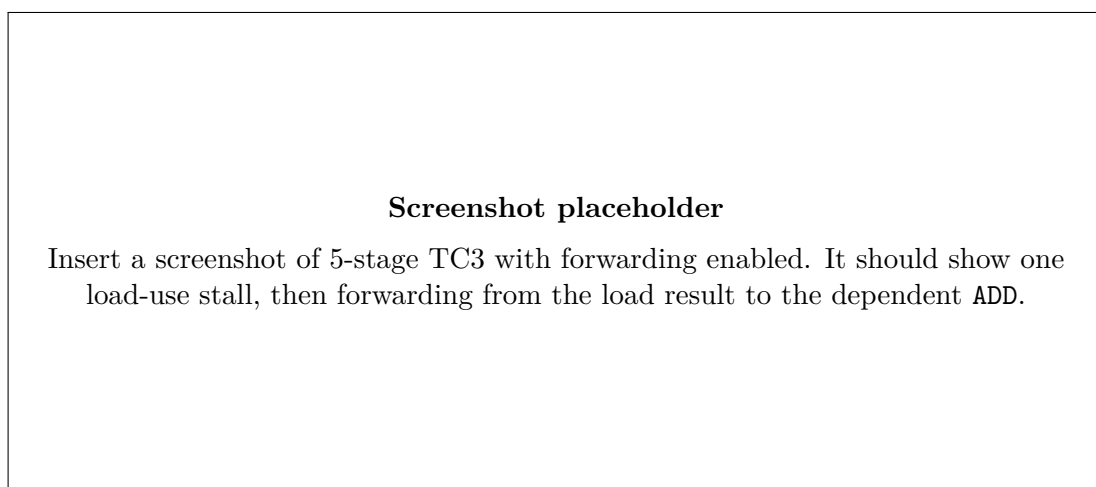


Figure 15: 5-stage TC3 load-use with forwarding.

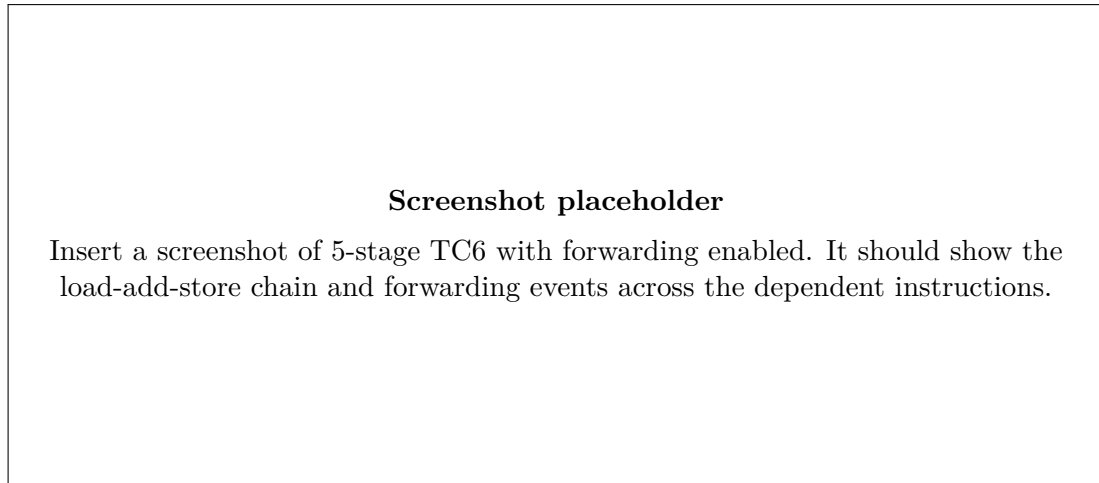


Figure 16: 5-stage TC6 load-add-store chain.

10 Limitations

The simulator intentionally stays within the assignment scope. It does not simulate:

- branch or jump instructions,
- control hazards,
- structural hazards,
- caches or memory latency,
- exceptions or interrupts,
- out-of-order execution,
- superscalar issue,
- multi-cycle functional units.

The applet is designed to make the behavior of simple in-order pipeline RAW hazards clear and visible rather than to reproduce every feature of a real processor.

11 Conclusion

The applet satisfies the assignment requirements for a compact interactive pipeline hazard simulator. It supports user-defined instruction sequences, both required pipeline configurations, cycle-by-cycle visualization, step and auto-run execution, reset behavior, RAW hazard detection, stall insertion, and forwarding visualization. The code is organized so that the simulation logic remains in `pipeline.js`, while rendering and interaction behavior are handled separately in `ui.js` and `style.css`.